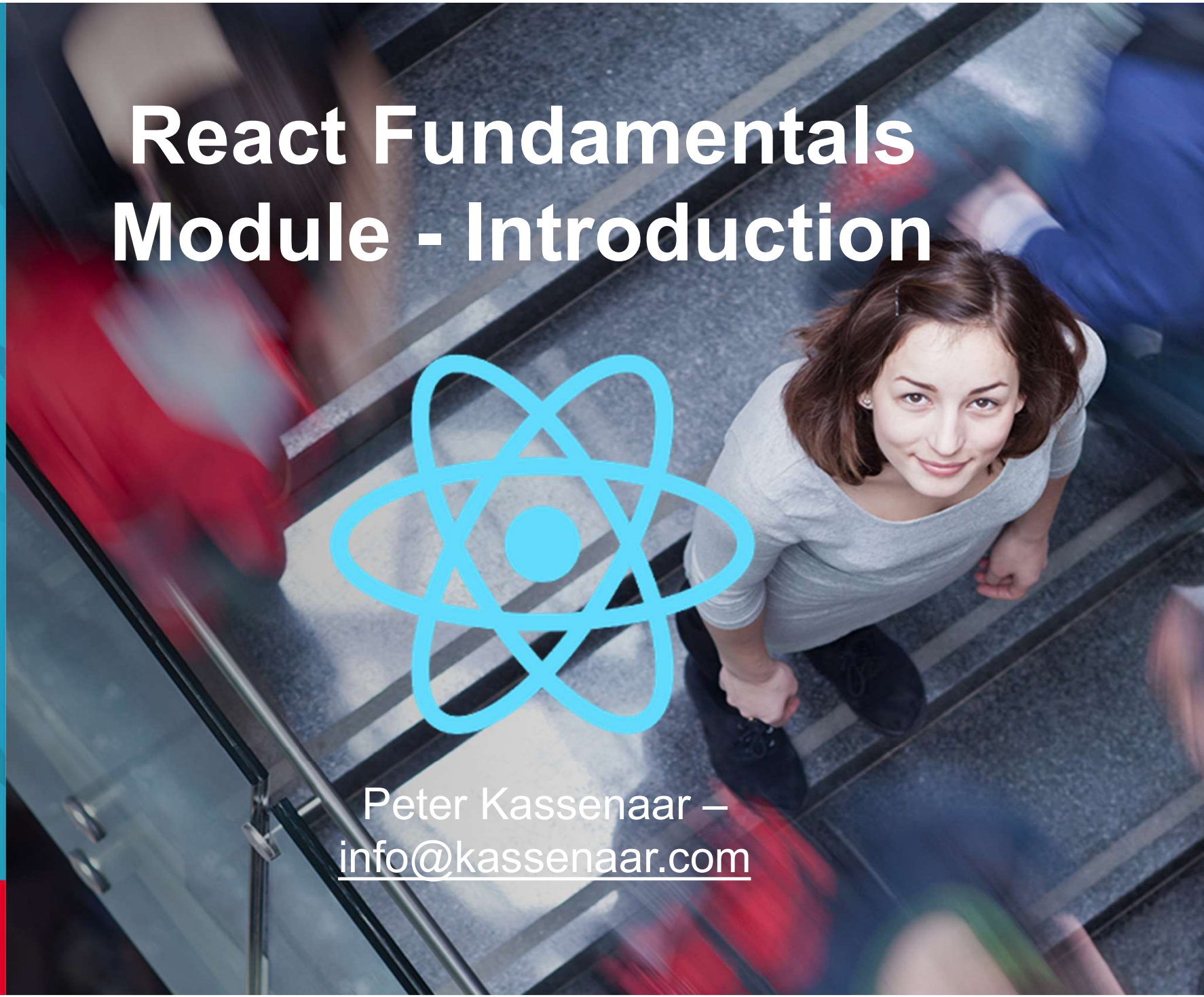
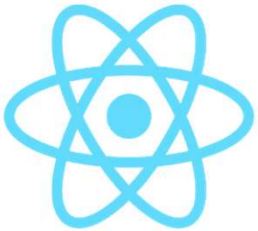
React Fundamentals Module - Introduction



Peter Kassenaar –
info@kassenaar.com

Goals of this module



*You're **not** going to be a React wizard in 2 days (sorry)*

but….

Goals

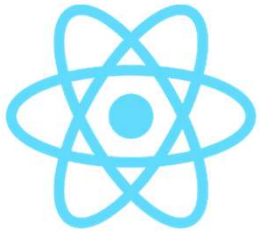
1. You *will* learn about the structure and architecture of React Apps. From a small `hello-world` app to the largest enterprise applications.

2. You are familiar with the main React concepts of the framework. You can always google the code details yourself.

3. You will have some hands-on experience regarding creating *React-apps* and components, API's/backends, routing and forms.

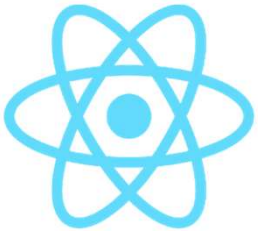
4. You can manage and maintain new and exsisting React apps. You know where to update code and what the basic principles are.

Agenda - 2 days



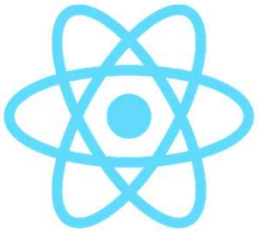
- Introduction – overview of the front-end landscape
- React tooling – installation, new project
- Hello World –the structure and architecture of React apps.
- Zooming in:
  - Components – Class based vs. Function based

Agenda – cont'd



- React hooks - introduction
- Data and state in components
- Communication between components
  - Props and events
- Handling user input
- Communicating with external API's
- Using React Router

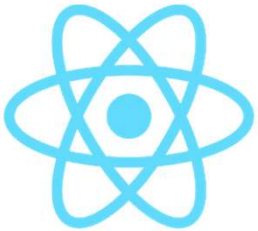
Labs and example code



1. Labs/Exercises
   - In the PDF's in the Github-repo. But: feel free to deviate. Adapt to suit your own needs! (hobby, work, current projects)

2. Example code
   - Executions of the exercises, small projects (`npm install`, `npm start`)
   - Work in progress – let me know of additions/errors!
   - github.com/PeterKassenaar/react-fundamentals

Advanced warning - First morning



Not so much code…

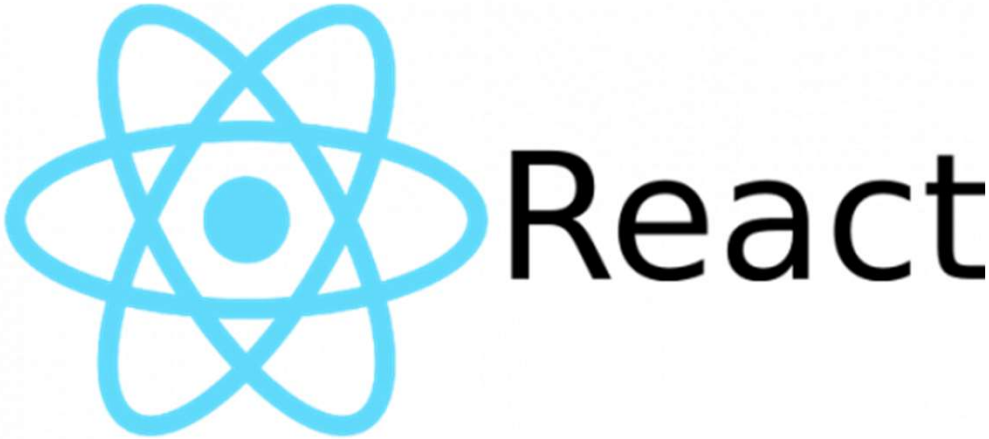
Concepts, architecture, structure

Questions?

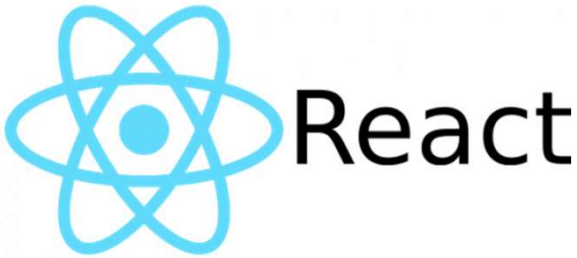
The Frontend Landscape

An overview of the frontend landscape

Front-end Frameworks – the big four



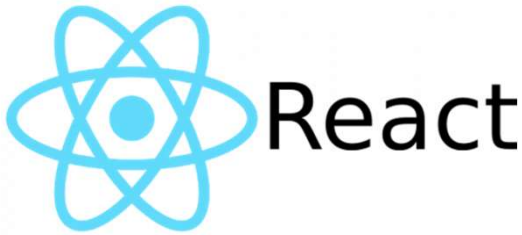
Similarities



- Creating Single Page Applications
- Based on components
- Data binding, props, events, routing, state management, …
- Huge ecosystem
- Huge community
- High adaptation rate

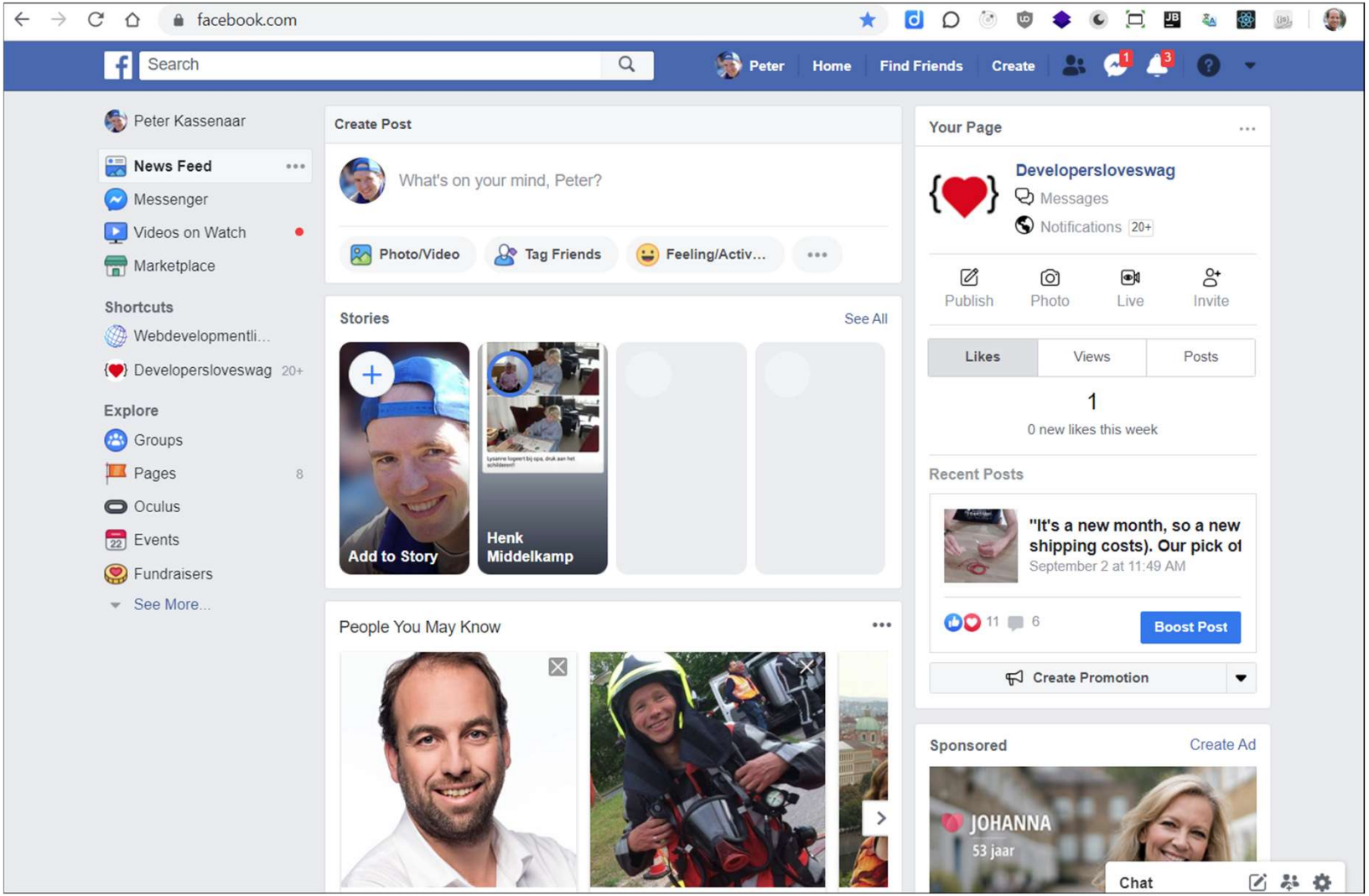
Differences (apart from syntax)

- Point of departure: HTML template, enhanced with framework specific tags and attributes
- One-stop-shop / solution

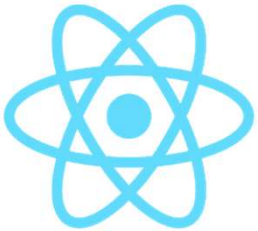


- Point of departure: JavaScript, JavaScript, JavaScript (JSX)
- Build-all-yourself / choice anxiety

React – Created by Jordan Walke (Facebook)



Brief history of React



2011 – Created at Facebook

2012 – Used on Instagram

2013 – Open source (May 29, 2013)

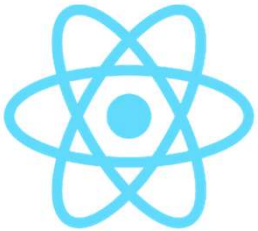
2014 – growth in popularity outside Facebook

2015 – React Native

2016 – React 15 (update to SemVer, from 0.14)

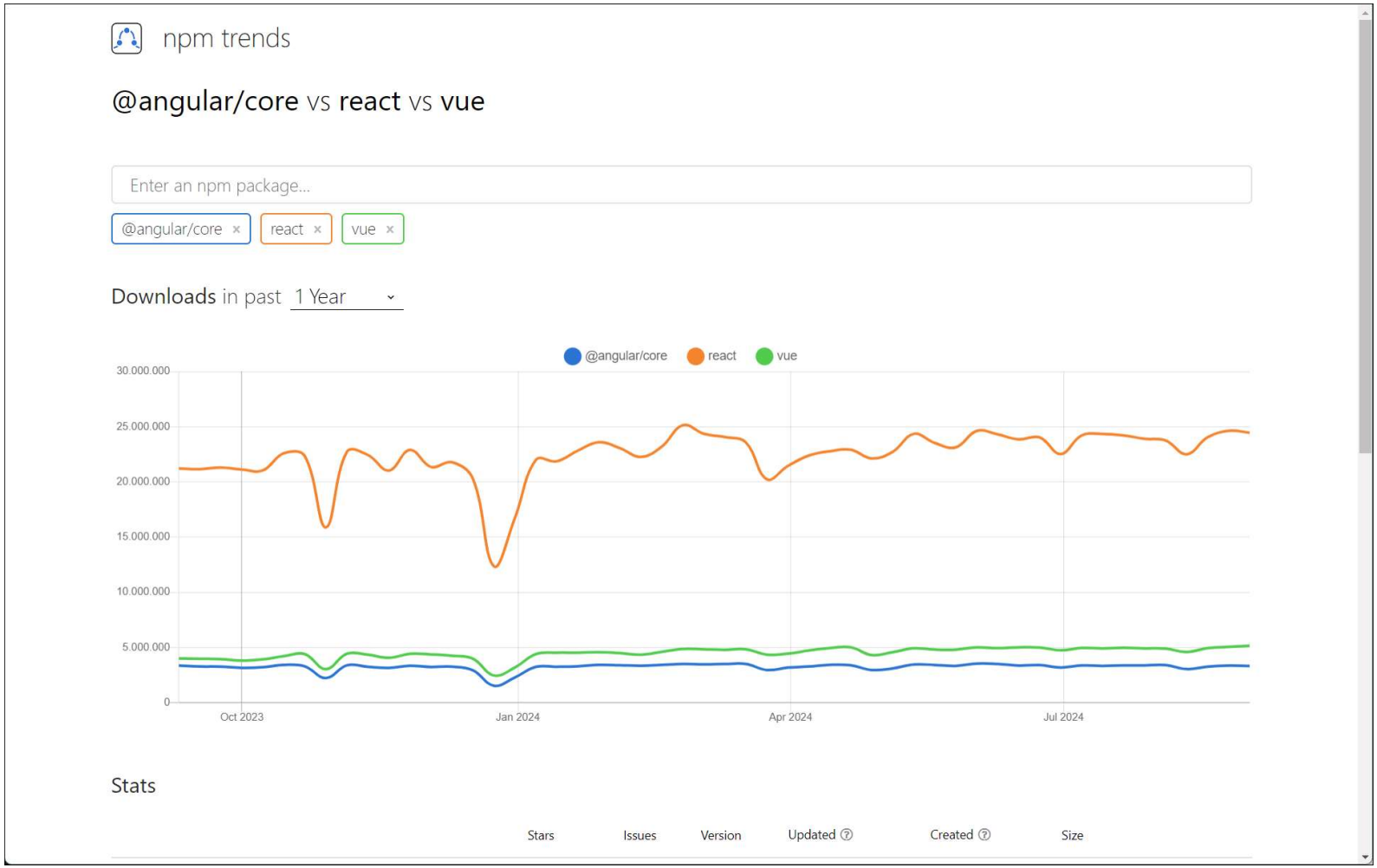
Today – *React 19, Facebook uses 30k+ components, employs full-time dev staff, used by many companies*

Why React?



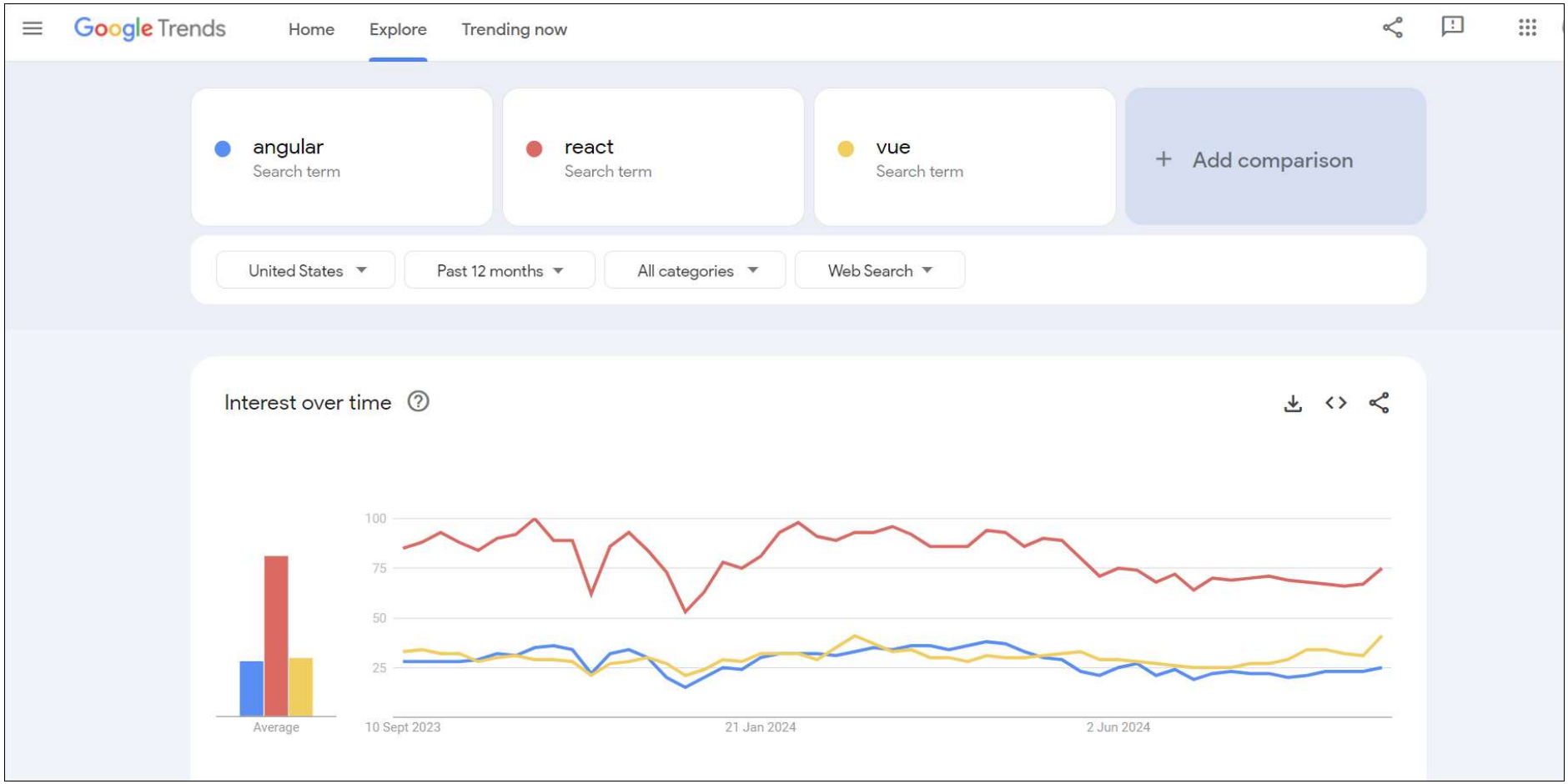
- Flexibility, *lots* of packages available
- Developer experience
- Large corporate investment by Facebook
- Large community (Github, Stack Overflow)
- Performance
- Testability

Npm trends comparison



https://www.npmtrends.com/@angular/core-vs-react-vs-vue

Google Trends



https://trends.google.com/trends/explore?geo=US&q=angular,react,vue

Angular vs React vs Vue: Which Framework to Choose

by Shaumik Daityari / November 30, 2023 / web & app frameworks

This post is a comprehensive guide on which is perhaps the right solution for you: **Angular vs React vs Vue**.

Just a couple of years ago, developers were mainly debating whether they should be using Angular vs React for their projects. But over the course of the last couple of years, we've seen a growth of interest in a third player called Vue.js.

If you are a developer starting out on a project and cannot decide on which JavaScript framework to use, this guide should help you make a decision.

Here we'll cover various aspects of Angular, Vue, and React to see how they suit your needs. This post is not just a guide on Angular vs React vs Vue but aims to provide a structure to help judge front-end JavaScript frameworks in general. In case a new framework arrives next year, you will know exactly what parameters to look at!

** In this post, we assume that you have basic knowledge of JavaScript and have used JavaScript frameworks as well.*

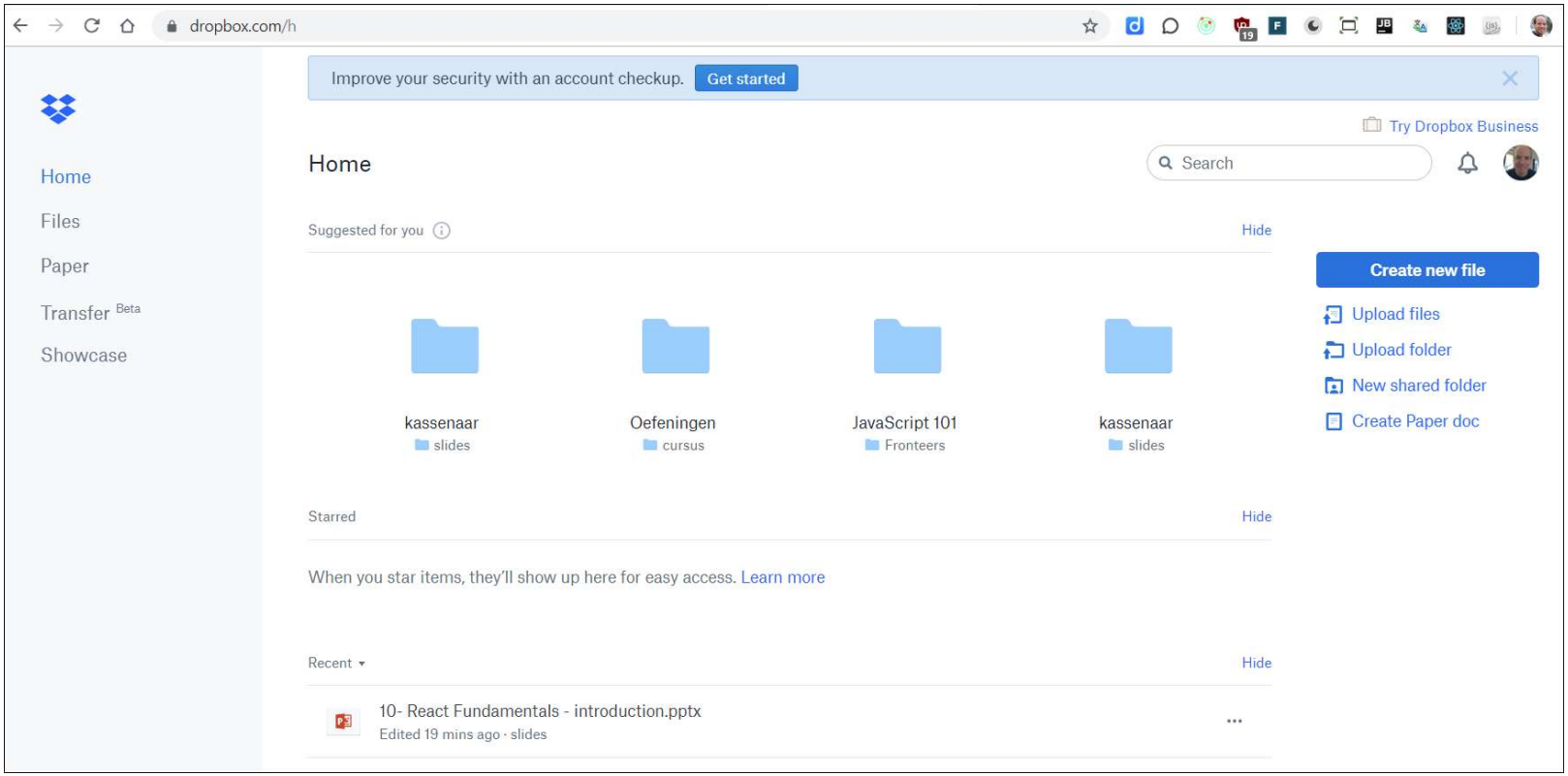
Let's get started:

– Table of contents:

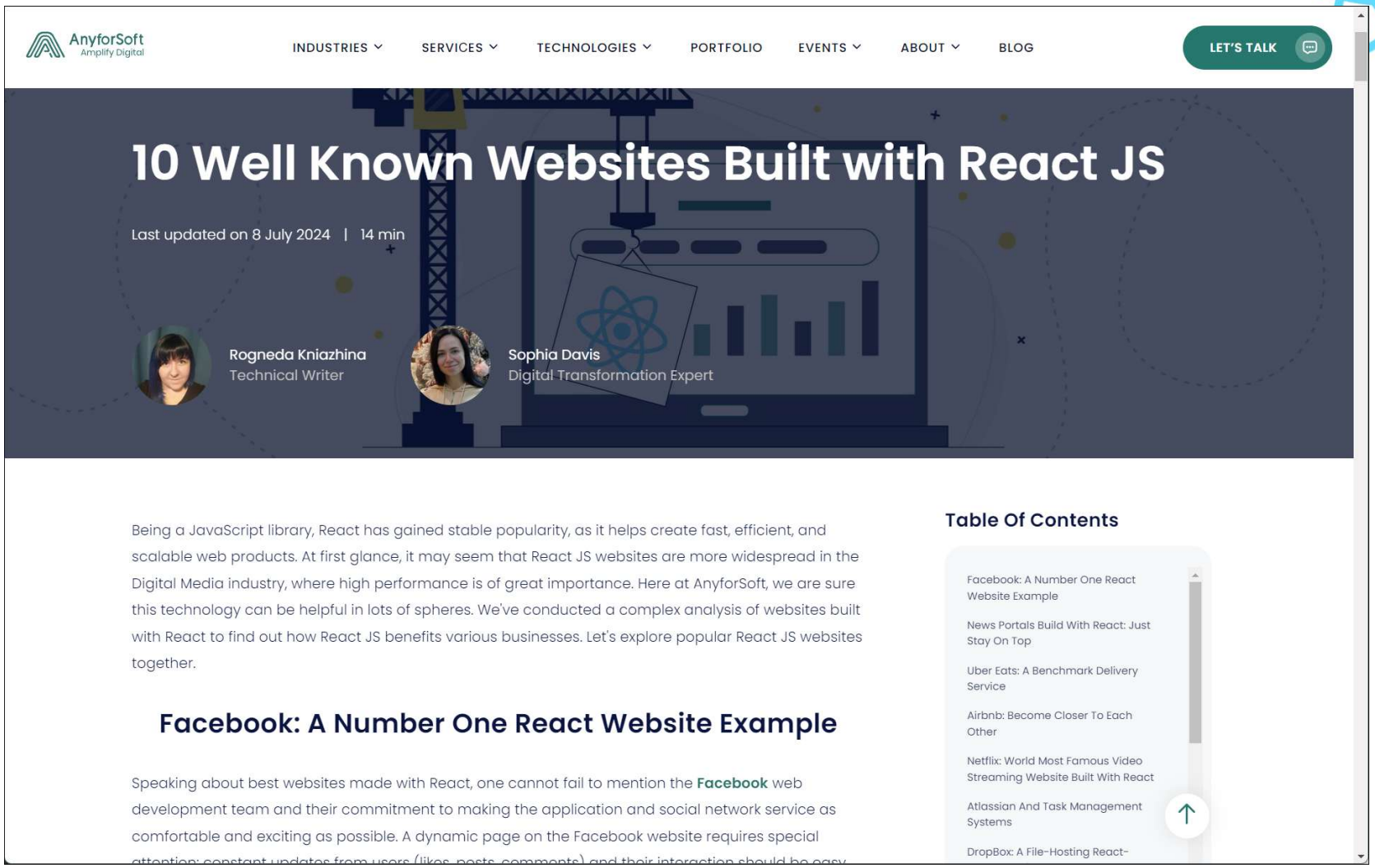
- Part 1: A brief history
  - How it all started | License | Popularity | Job market
- Part 2: Community and development
- Part 3: Migrations

https://www.codeinwp.com/blog/angular-vs-vue-vs-react/

“React is used by many Fortune 500 companies, including Airbnb, BBC, Instagram and Dropbox”



Google for ‘built with react’



AnyforSoft
Amplify Digital

INDUSTRIES SERVICES TECHNOLOGIES PORTFOLIO EVENTS ABOUT BLOG LET’S TALK

10 Well Known Websites Built with React JS

Last updated on 8 July 2024 | 14 min

Rogneda Kniazhina
Technical Writer

Sophia Davis
Digital Transformation Expert

Being a JavaScript library, React has gained stable popularity, as it helps create fast, efficient, and scalable web products. At first glance, it may seem that React JS websites are more widespread in the Digital Media industry, where high performance is of great importance. Here at AnyforSoft, we are sure this technology can be helpful in lots of spheres. We’ve conducted a complex analysis of websites built with React to find out how React JS benefits various businesses. Let’s explore popular React JS websites together.

Facebook: A Number One React Website Example

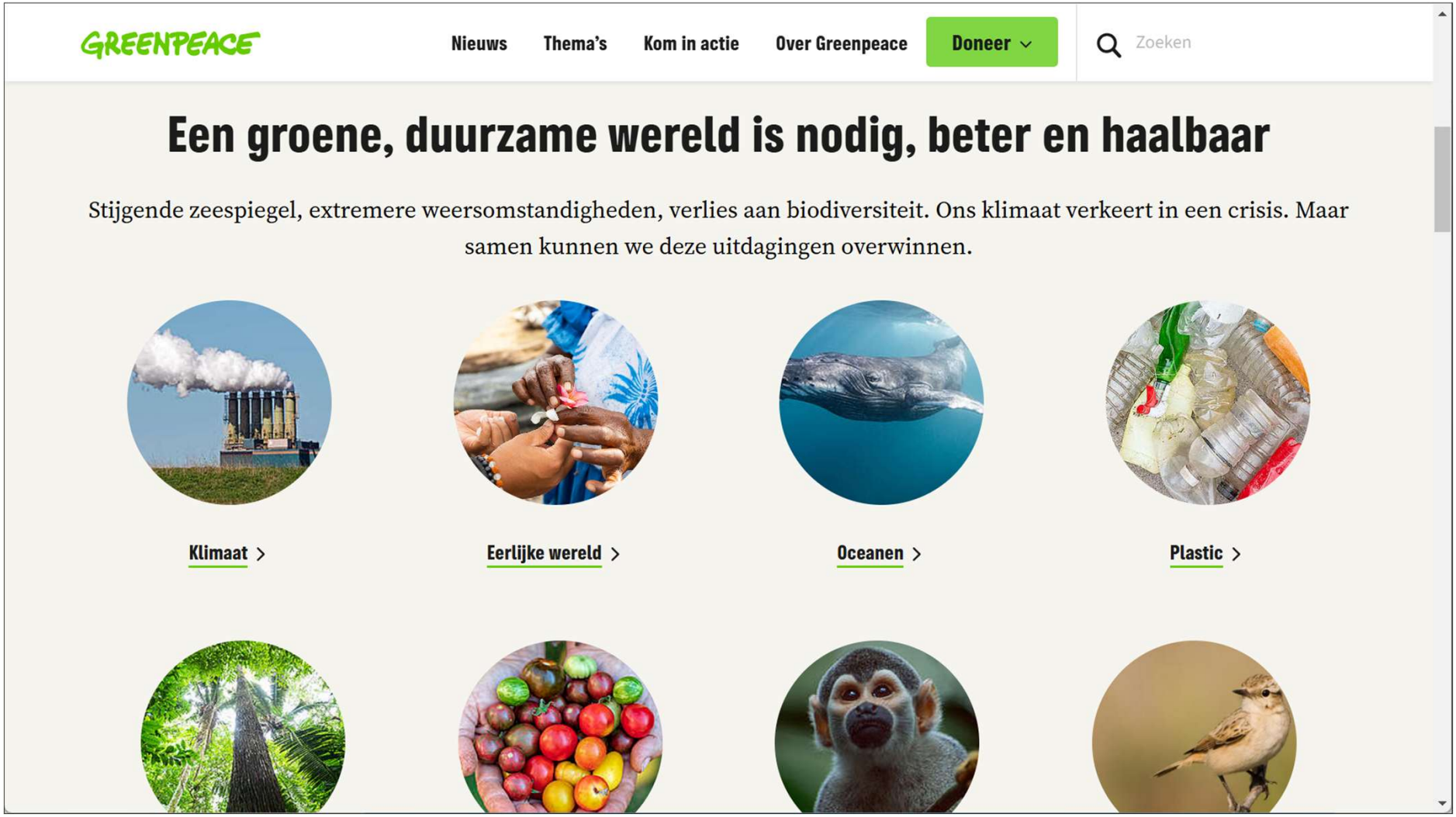
Speaking about best websites made with React, one cannot fail to mention the **Facebook** web development team and their commitment to making the application and social network service as comfortable and exciting as possible. A dynamic page on the Facebook website requires special attention: constant updates from users (likes, posts, comments) and their interaction should be easy

Table Of Contents

Facebook: A Number One React Website Example
News Portals Build With React: Just Stay On Top
Uber Eats: A Benchmark Delivery Service
Airbnb: Become Closer To Each Other
Netflix: World Most Famous Video Streaming Website Built With React
Atlassian And Task Management Systems
DropBox: A File-Hosting React-

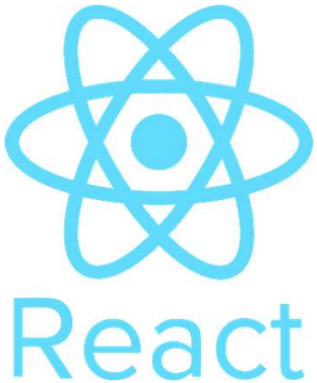
https://anyforsoft.com/blog/10-famous-websites-built-react-js

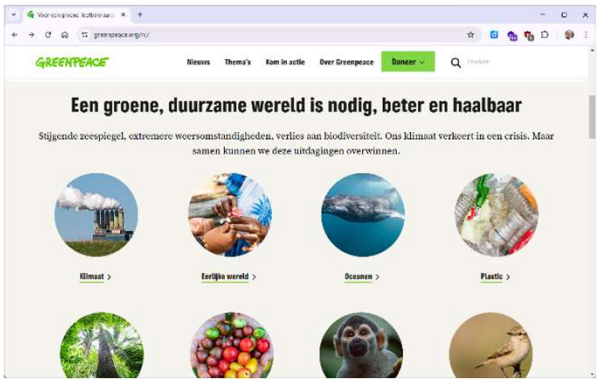
NL:



www.greenpeace.org/nl/

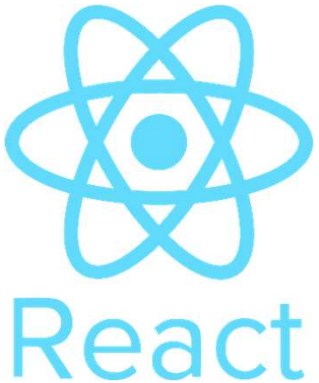
The React build process - classic

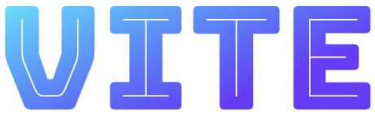


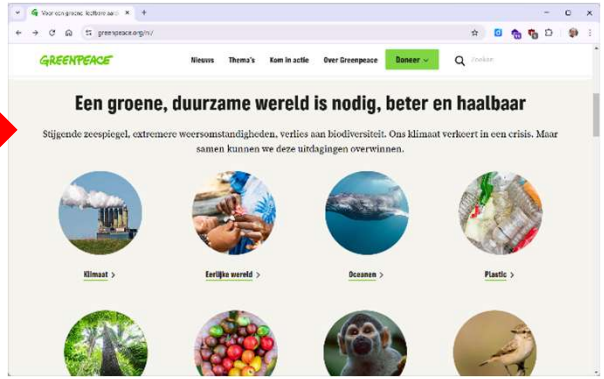


Developer → build → browser

The React build process – as of 2025

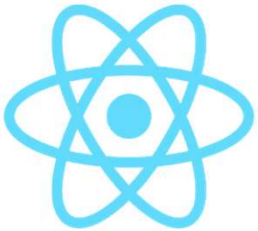






Developer → build → browser

Concerns on using React



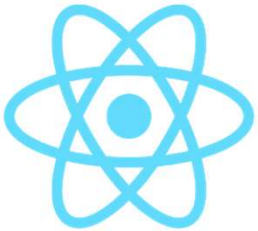
"JSX is so different from HTML!"

- True, but that's the choice React made
- Point of departure is JavaScript. Not the HTML-template

```
<h2 color="red">This is a heading</h2>
```

```
React.createElement('h2', { color: 'red'}, 'This is a heading');
```

JSX differences

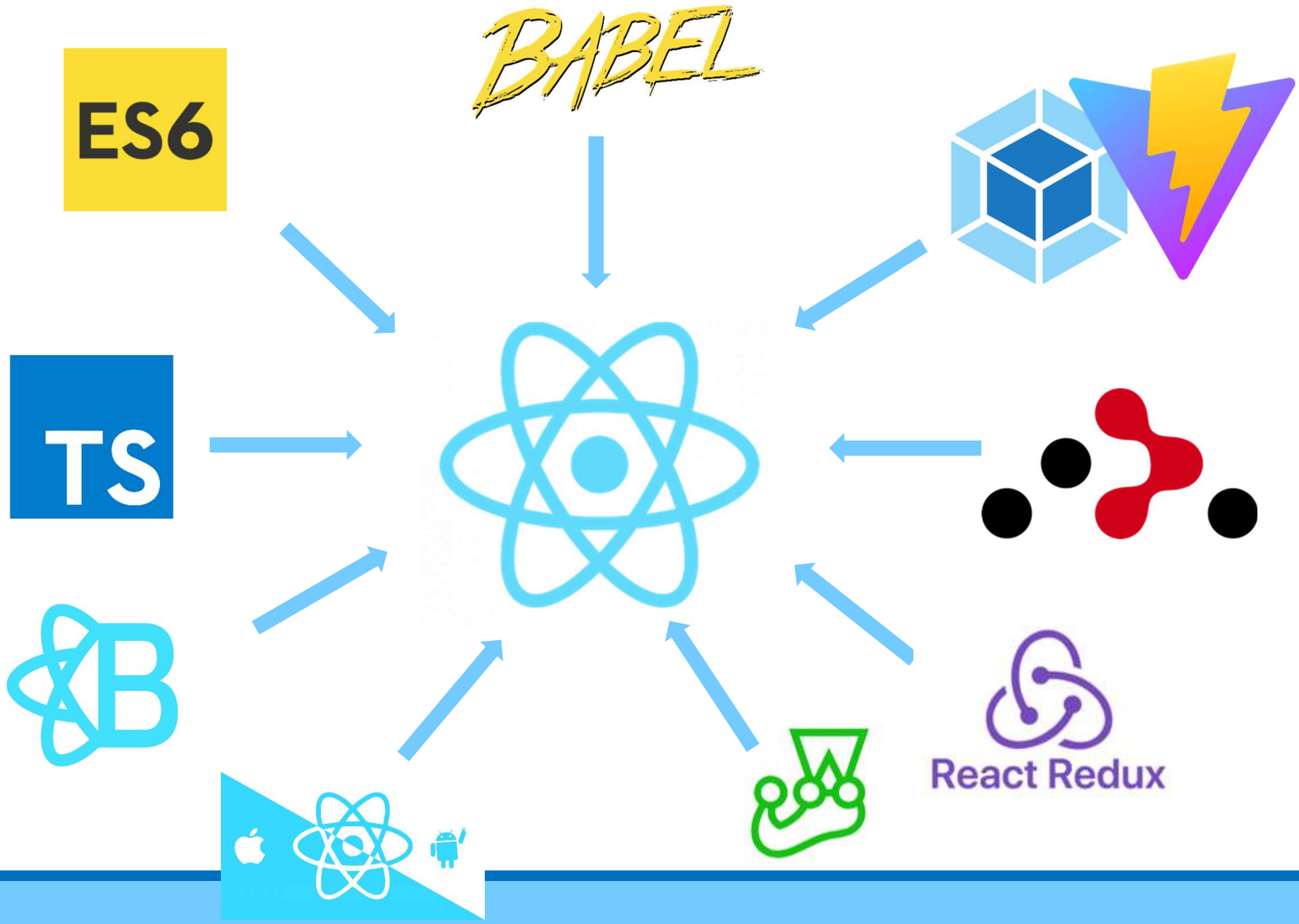


However - JSX looks a lot like HTML. There are only a few differences:

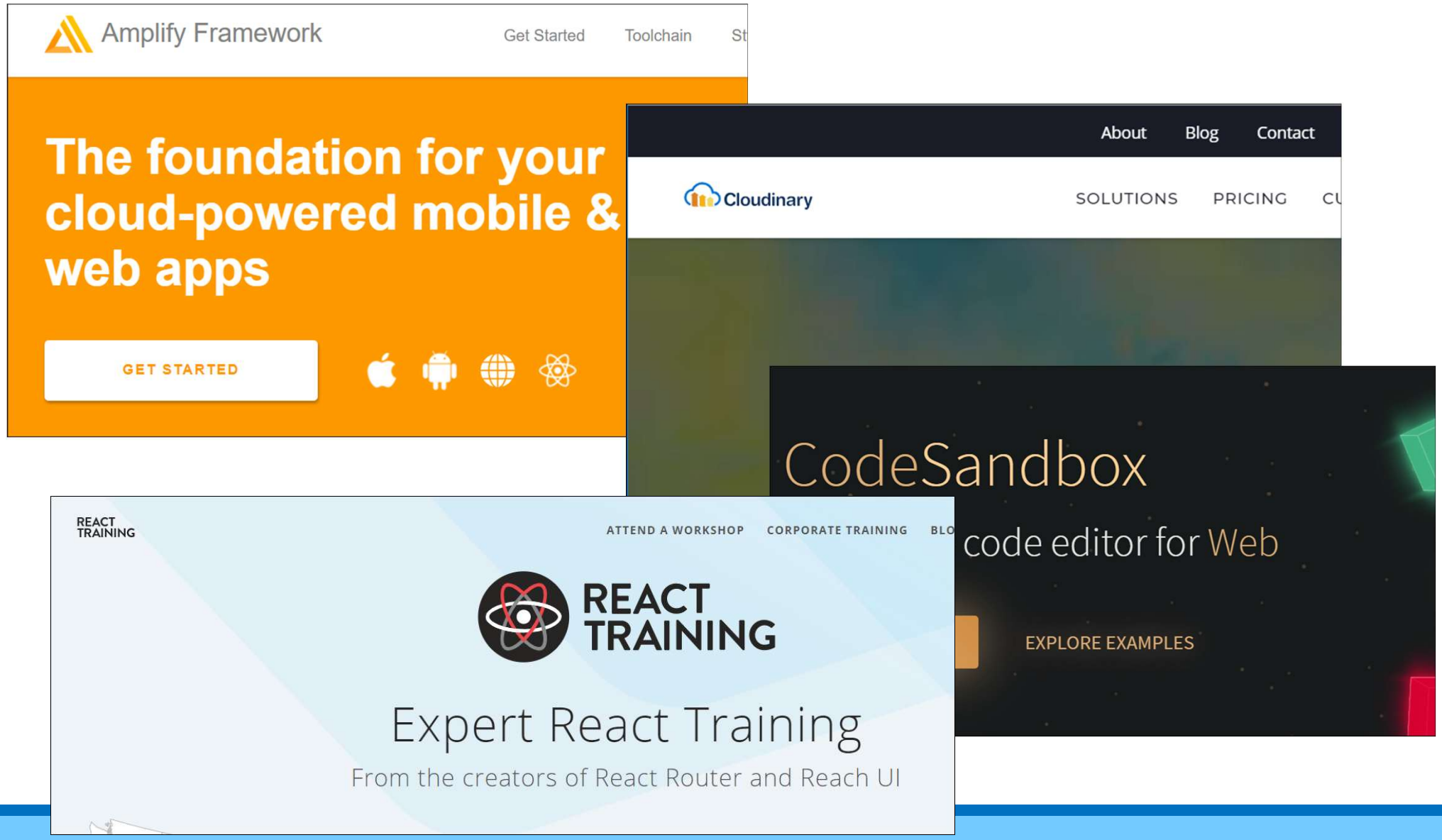
HTML	JSX
`for` (for labels)	`htmlFor`
`class`	`className`
`style="color: red"`	`<style={{color: 'red'}} >`
`<!-- comment -->`	`{/* Comment */}`
Events: `click`, `blur`, `focus` etc	Prefix with on+capital: `onClick`, `onBlur`, `onFocus`, etc

The React ecosystem

React is not isolated/not living on an island


ES6
BABEL
TS
React Redux

Additional services, options, etc.



H.U.G.E community

fezvrasta published 2.2.5 • 12 days ago

react-dropzone

Simple HTML5 drag-drop zone with React.js

react-component react drag drop upload

rolandjitsu published 11.3.2 • 11 days ago

react-color

A Collection of Color Pickers from Sketch, Photoshop, Chrome & more

react color picker react-component colorpicker picker sketch chrome photoshop material design popup

case published 2.19.3 • 5 months ago

react-scripts

Configuration and scripts for Create React App.

iansu published 4.0.3 • a month ago

react-table

Hooks for building lightweight, fast and extendable datagrids for React

react table react-table datagrid

tannerlinsley published 7.6.3 • 3 months ago

react-docgen

A CLI and toolkit to extract information from React components for documentation generation.

https://www.npmjs.com/search?q=react

Beware: your own package.json might be like…

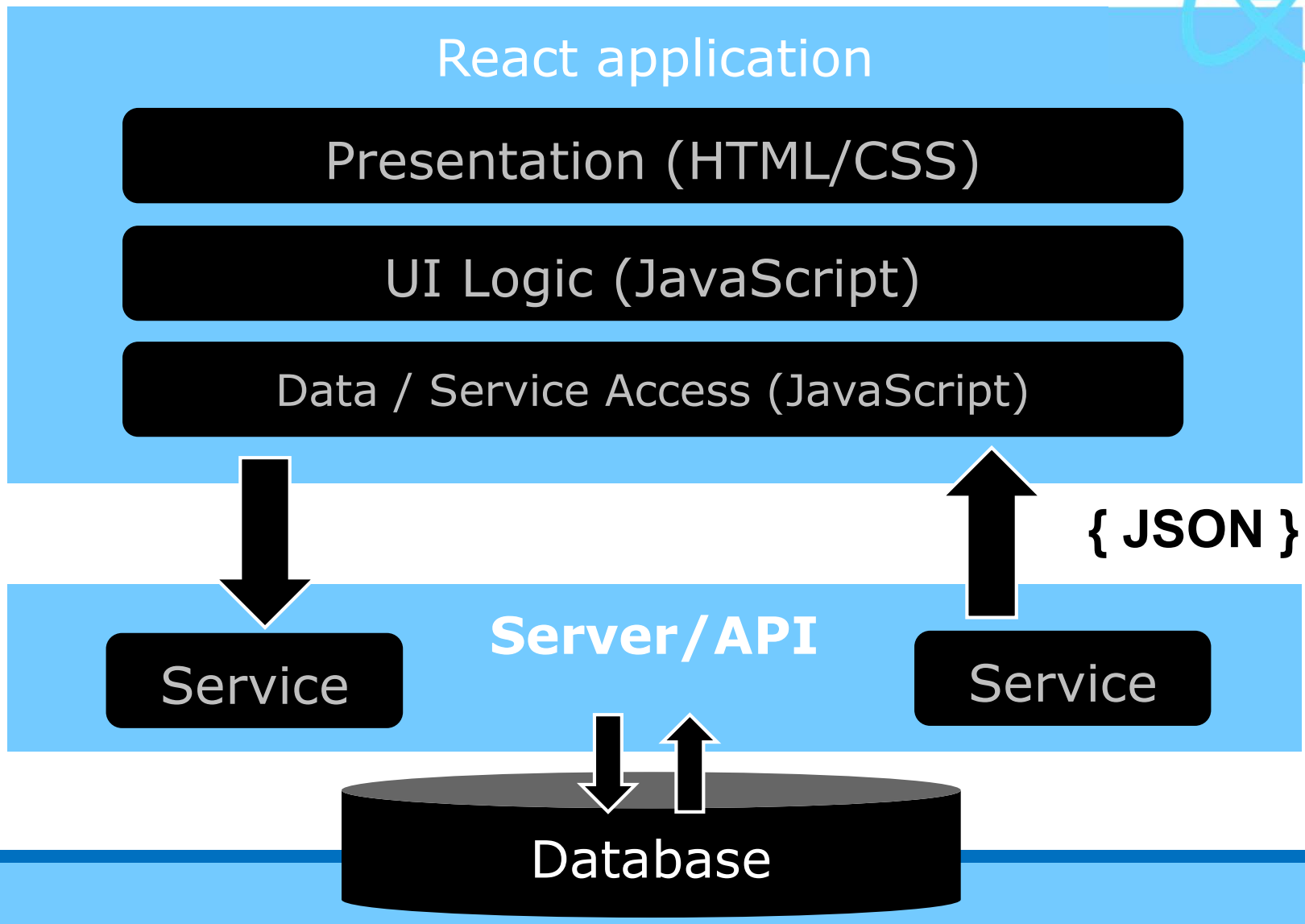
```
"dependencies": {
    "bootstrap": "^4.3.1",
    "connected-react-router": "6.5.2",
    "emotion": "^9.2.12",
    "emotion-server": "^9.2.12",
    "faker": "^4.1.0",
    "history": "4.10.1",
    "interactjs": "1.3.4",
    "jquery": "^3.4.1",
    "jquery-validation": "latest",
    "jquery-validation-unobtrusive": "latest",
    "merge": "1.2.1",
    "moment": "^2.26.0",
    "popper.js": "^1.16.0",
    "randomcolor": "^0.5.4",
    "react": "16.11.0",
    "react-calendar-timeline": "0.27.3-beta",
    "react-confirm-alert": "^2.6.2",
    "react-contexify": "^4.1.1",
    "react-dom": "16.11.0",
    "react-emotion": "^9.2.12",
    "react-helmet": "^5.2.0",
    "react-icons": "^3.11.0",
    "react-loader-spinner": "^3.1.14",
    "react-new-window": "^0.1.2",
    "react-redux": "7.1.1",
    "react-router": "5.1.2",
    "react-router-dom": "5.1.2",
    "react-scripts": "^3.2.0",
    "react-select": "^3.1.0",
    "react-split-it": "^1.1.3",
    "react-toastify": "^6.0.8",
    "react-tooltip": "^4.2.9",
    "react-vis": "1.11.7",
    "reactstrap": "^8.5.1",
    "reactstrap-confirm": "^1.2.0",
    "redux": "4.0.4",
    "redux-thunk": "2.3.0",
    "styled-components": "^4.0.0",
    "svgo": "1.3.0"
  }
```

And more….

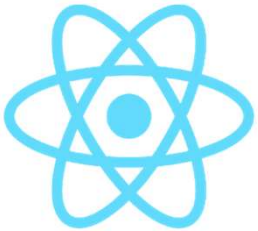
React’s basic concepts

Getting a conceptual understanding of every React app

Single Page Application



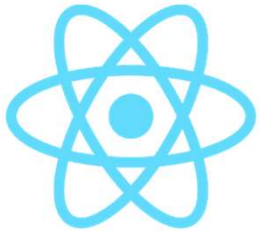
3 Basic principles in every React App



1. Components

- Like functions
  - `class` based and `function` based components
  - Wild debate…
- Input: `props` and `state`
- Output: UI (via the `render()` or `return()` function)
- Reusable and composable
- Mentioned via their name, like `<MyComponent />`
- Components can have private state

3 Basic principles in every React App



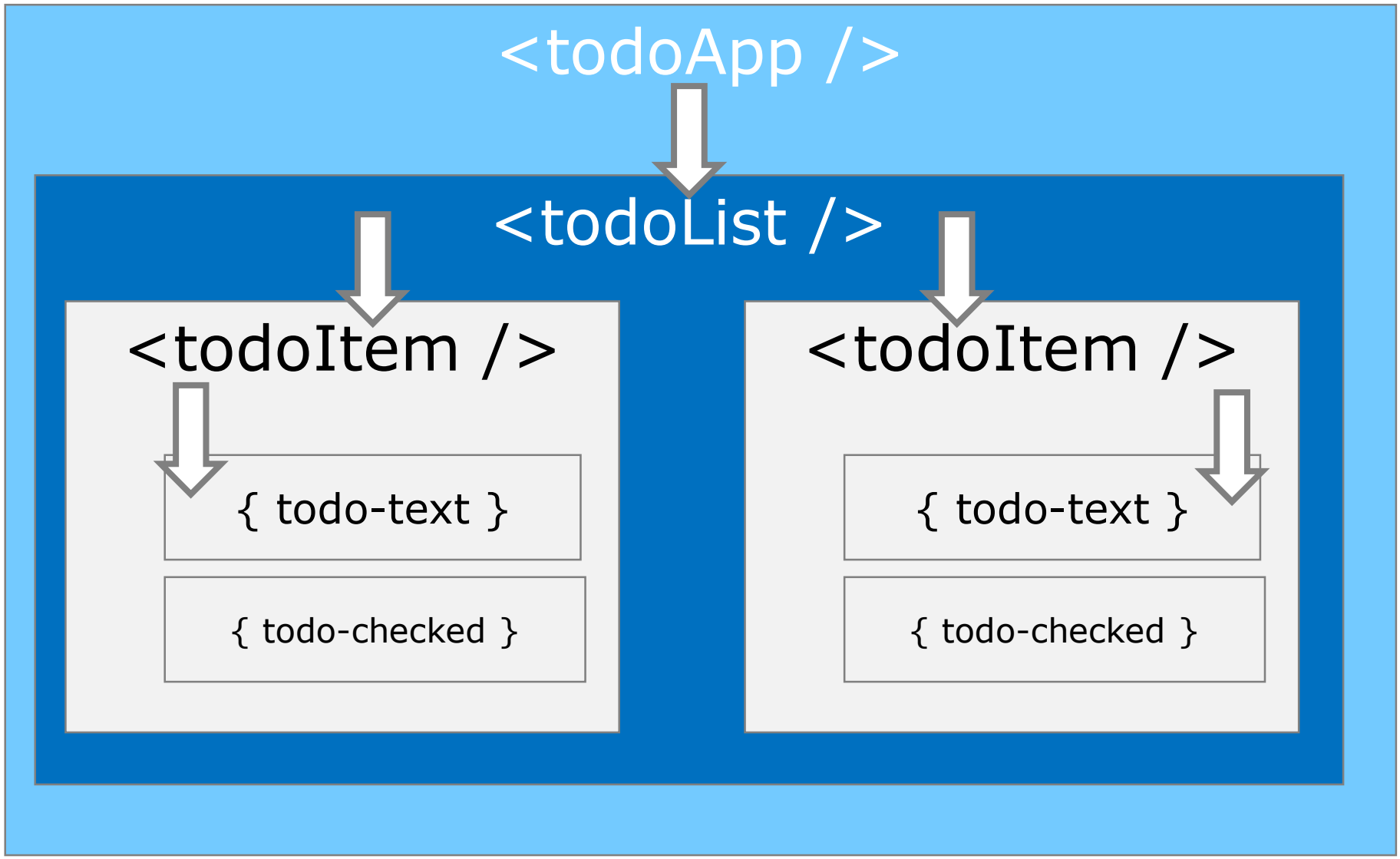
2. Reactive updates

- React will react to the changes it makes or receives
- Update the browser, based on those changes

3. Virtual views in memory

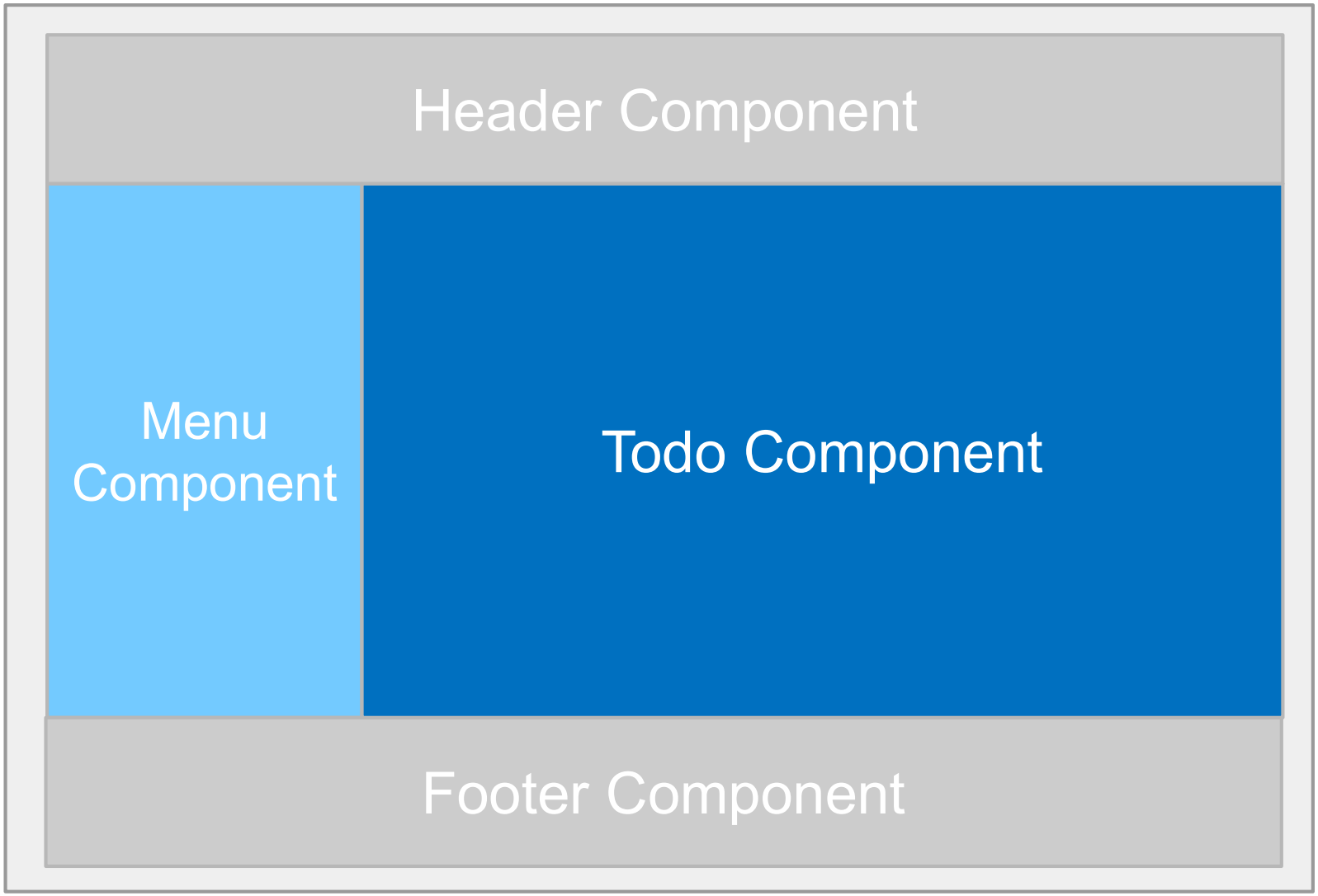
- Don’t write HTML (for instance to handle data)
- Use JavaScript (or JSX) to generate HTML
- No HTML template language
- Virtual DOM or Tree reconciliation
- Build the DOM in-memory before adding it to the UI

React – app- & component structure



"A React app is a tree of components"

Components – visual representation



React components

Function based components

Simple – just write functions that return the UI for the component

Class based components

A bit more complex – but also more powerful *(? - opinion)*

You will use – and need to learn – both

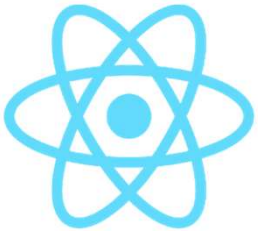
```
const Counter = props => {
    const [counter, setCounter] = useState(0)
    return (
        <div>
            <h1>A counter component</h1>
        </div>
    )
};
```

Function component

Class component

```
class Counter extends React.Component {
    state = {
        counter: 0
    };

    render() {
        return (
            <div>
                <h1>A counter component</h1>
            </div>
        )
    }
};
```

Compiling a class component

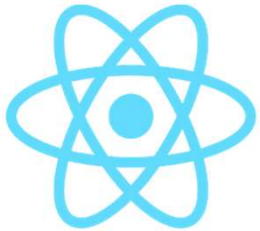


JSX is compiled to JavaScript

(here: the class component):

```
class Counter extends React.Component {
    state = {
        counter: 0
    };

    render() {
        return (
            React.createElement('div', null,
                React.createElement('h1', null, 'A counter component')
            )
        )
    }
};
```

Current state:



Until ca. 2015 : `functions`

Ca. 2015 – 2020: `functions` + `classes`

From ca. 2020 – again more focus on `functions` and *hooks*

Verdict:

Existing applications and libs – you *will* encounter classes

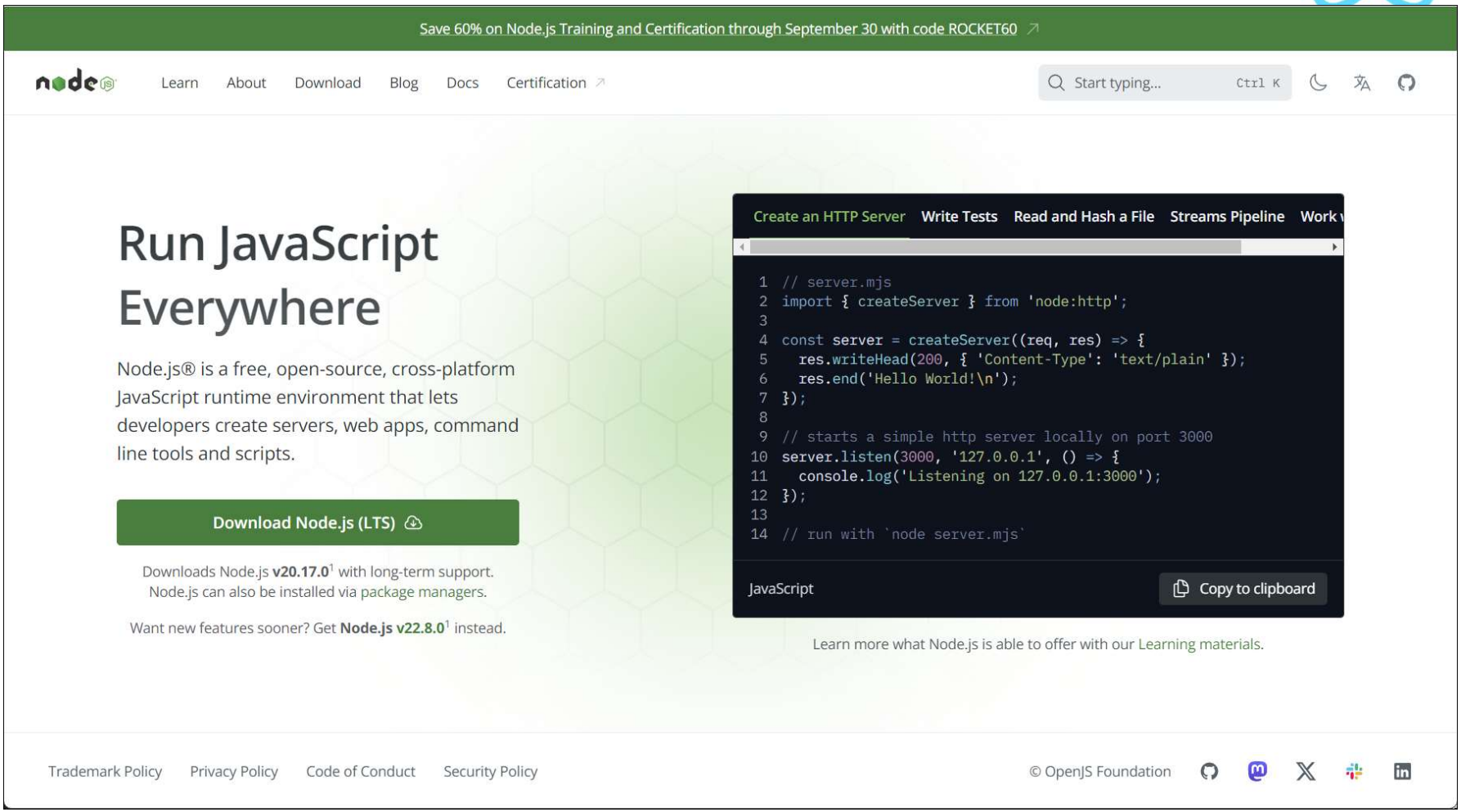
New applications – using Function Based Components is the way to go

Questions?

Let’s create an app

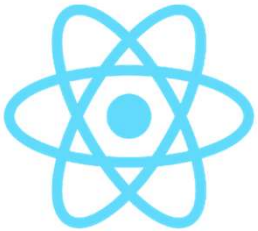
Install tooling and create your first app

NodeJS



https://nodejs.org/en/

Node.js recommendations



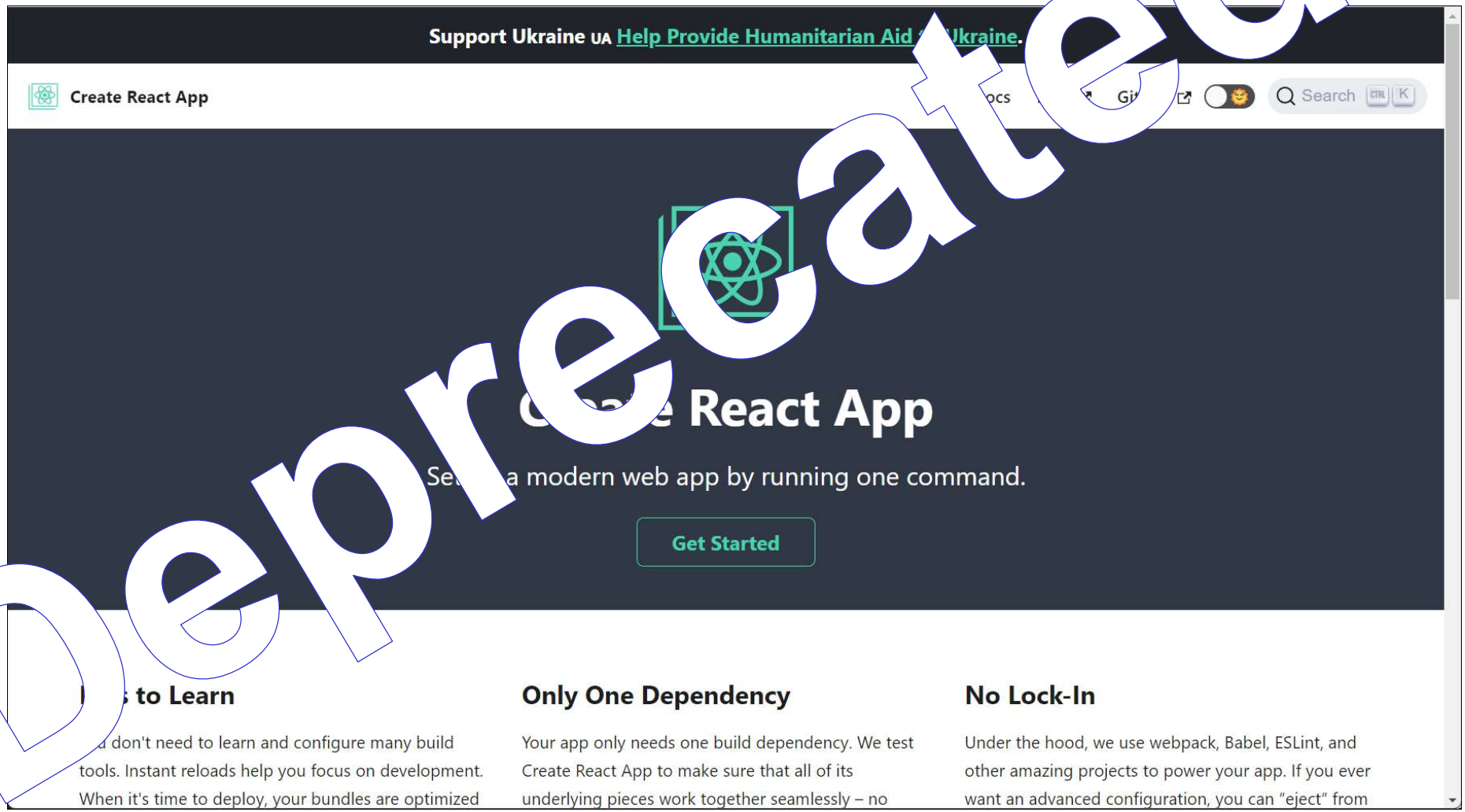
- Use an even version number (e.g. Node 16, 18, 20)
  - Only even version numbers retain LTS-status – e.g. are more supported in the long run/future
- Odd numbers are mainly for testing latest features
  - Node 17, 19, 21 etc.
  - If features are OK, they are transfered to an even LTS-version

OLD: `create-react-app`

- Create React App (or ‘CRA’ for short)
  - Scaffold new projects
  - Set up JSX compilation by Babel
  - Set up Webpack bundling
  - Set up Linting
  - …
- All this is done by CRA + webpack
- https://create-react-app.dev/

```
npx create-react-app my-app
```

create-react-app.dev



Deprecated

NEW: `vite`-based projects

- `npm create vite@latest`
  - Optional: use command line parameters, OR
  - Use terminal prompt

```
C:\WINDOWS\system32\cmd.

PS C:\Users\Gebruiker\Desktop> npm create vite@latest
Need to install the following packages:
  create-vite@6.3.1
Ok to proceed? (y) y

◇  Project name:
   react-vite

◆  Select a framework:
   ○ Vanilla
   ○ Vue
   ● React
   ○ Preact
   ○ Lit
   ○ Svelte
   ○ Solid
   ○ Qwik
   ○ Angular
   ○ Others
```

```
◇  Select a framework:
   React

◆  Select a variant:
   ○ TypeScript
   ○ TypeScript + SWC
   ● JavaScript
   ○ JavaScript + SWC
   ○ React Router v7 ↗
```

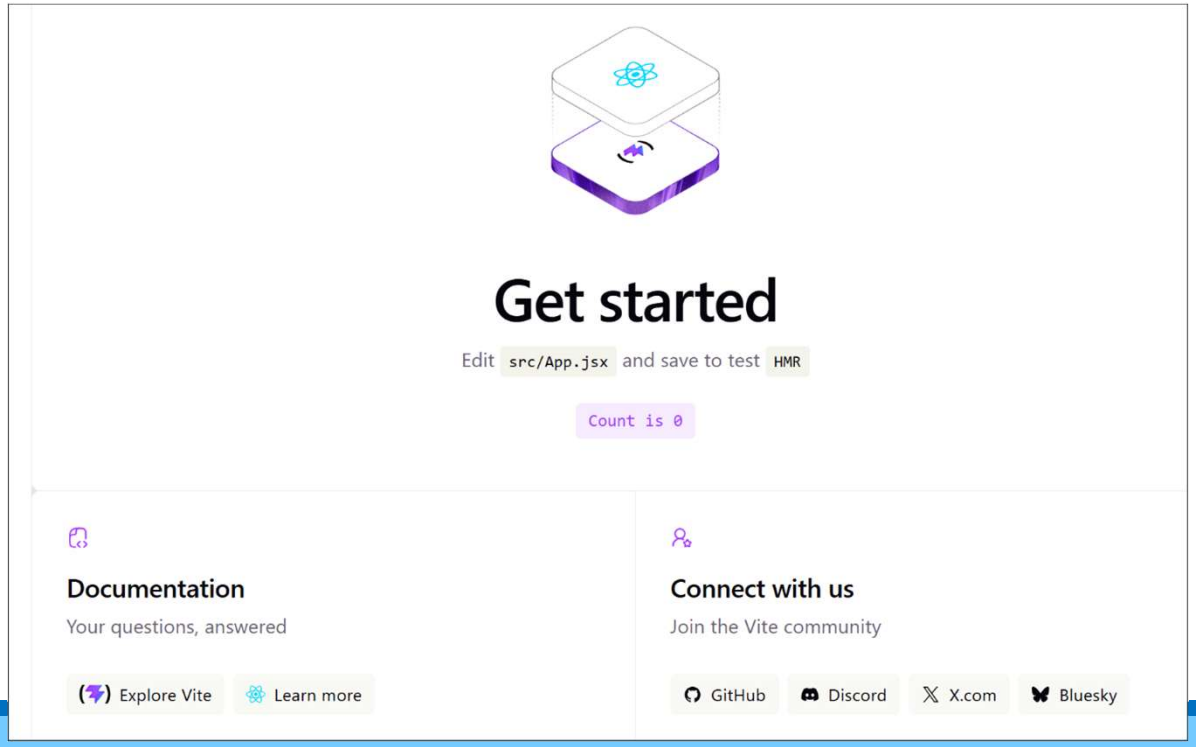
Go to directory + install

- cd <your-directory>
- npm install
- npm run dev

```
npm notice
PS C:\Users\Gebruiker\Desktop> cd .\react-vite\
PS C:\Users\Gebruiker\Desktop\react-vite> npm i

added 148 packages, and audited 149 packages in 8s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Gebruiker\Desktop\react-vite>
```

Running the default project (NEW)

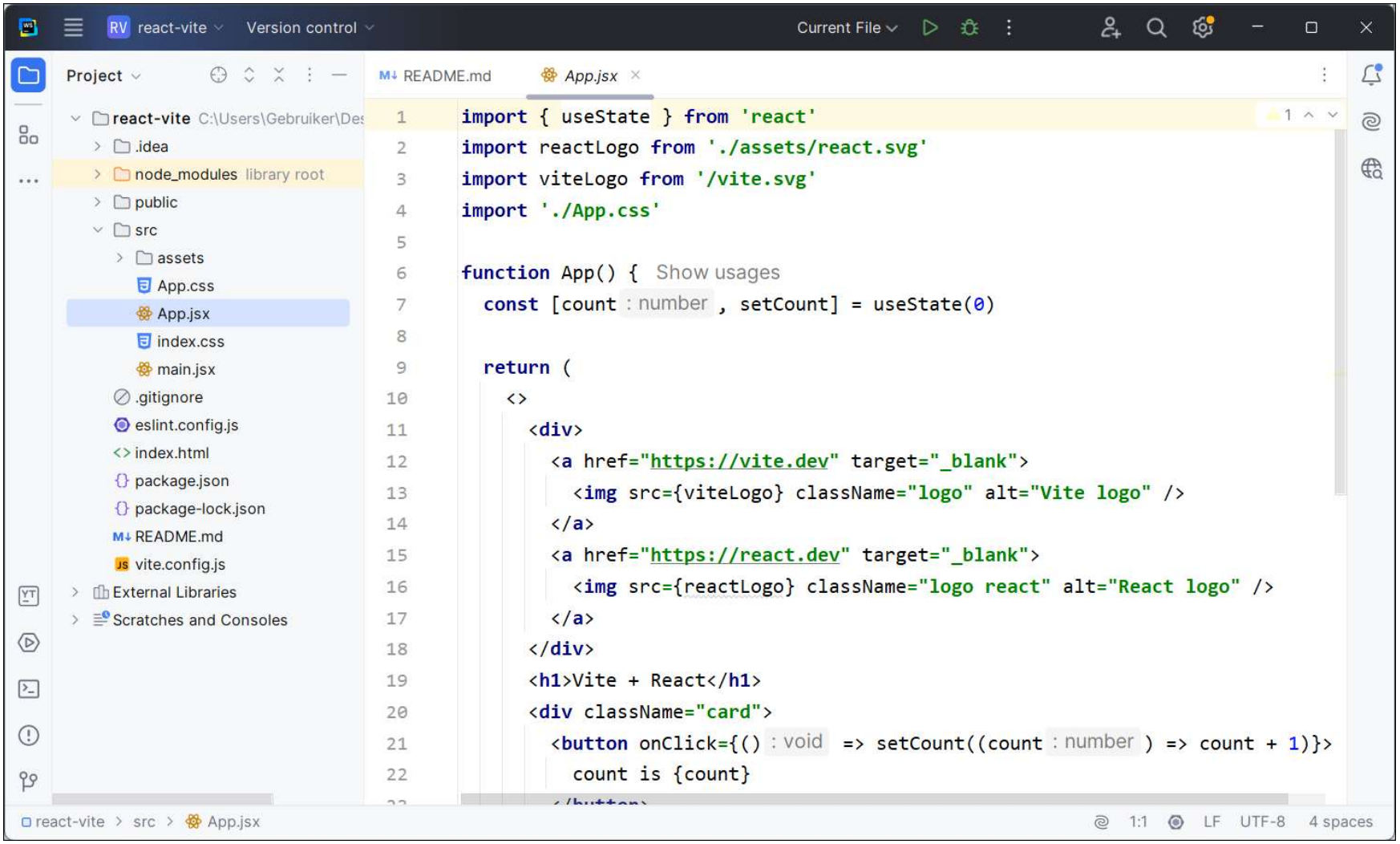
- `npm run dev`
- http://localhost:5173



Running the default project (OLD)

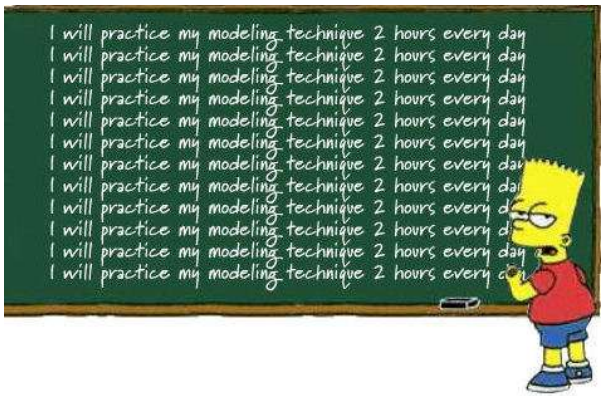
- `npm start`
- http://localhost:3000

File structure



Mini workshop

- Create a default project `npm create vite@latest`
  - https://react.dev/learn/build-a-react-app-from-scratch (background info)
- Create a new, blank React project
- Run it:
  - `npm run dev`
- Replace some content in `src/App.jsx` component with your own code.
- Code example: ../100-helloworld



Let’s look at the code

A look inside the key files in the generated code

#1. Package.json

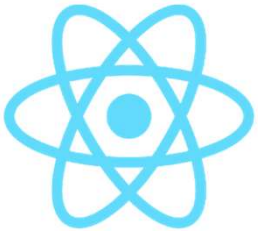
M↓ README.md {} package.json ×

```
1  {
2    "name": "react-vite",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "lint": "eslint .",
10     "preview": "vite preview"
11   },
12   "dependencies": {
13     "react": "^19.0.0",
14     "react-dom": "^19.0.0"
15   },
16   "devDependencies": {
17     "@eslint/js": "^9.21.0",
18     "@types/react": "^19.0.10",
19     "@types/react-dom": "^19.0.4",
20     "@vitejs/plugin-react": "^4.3.4",
21     "eslint": "^9.21.0",
22     "eslint-plugin-react-hooks": "^5.1.0",
```

#2 ./src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`main.jsx` (previously: `index.js`)



- Is the root of your application
- Imports starting component
  - Defaults to: `<App />`
- Adds `App` to the root element in `index.html`
- Typically you *never touch* this file again

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

#3. `App.jsx`

- `<App />` is the main component of the app.
  - You typically replace the contents with your own code
  - Main Navigation, routing, other content, footer, etc.
- You add (e.g. `import`) new components to `<App />`
  - React builds its internal graph based on the contents of `<App />`

```
import React from 'react';
import logo from ./logo.svg';
import ./App.css';
```

Import(s)

```
function App() {
  return (
    <div className="App">
      <header className="App-header">
        …
      </header>
    </div>
  );
}
```

Component (`function` or `class`)

```
export default App;
```

Export

#4. Other files

- `*.css` – various stylesheets.
- `*.test.js` – various unit testing files.
- Folder `/public` – all files that you want to publish

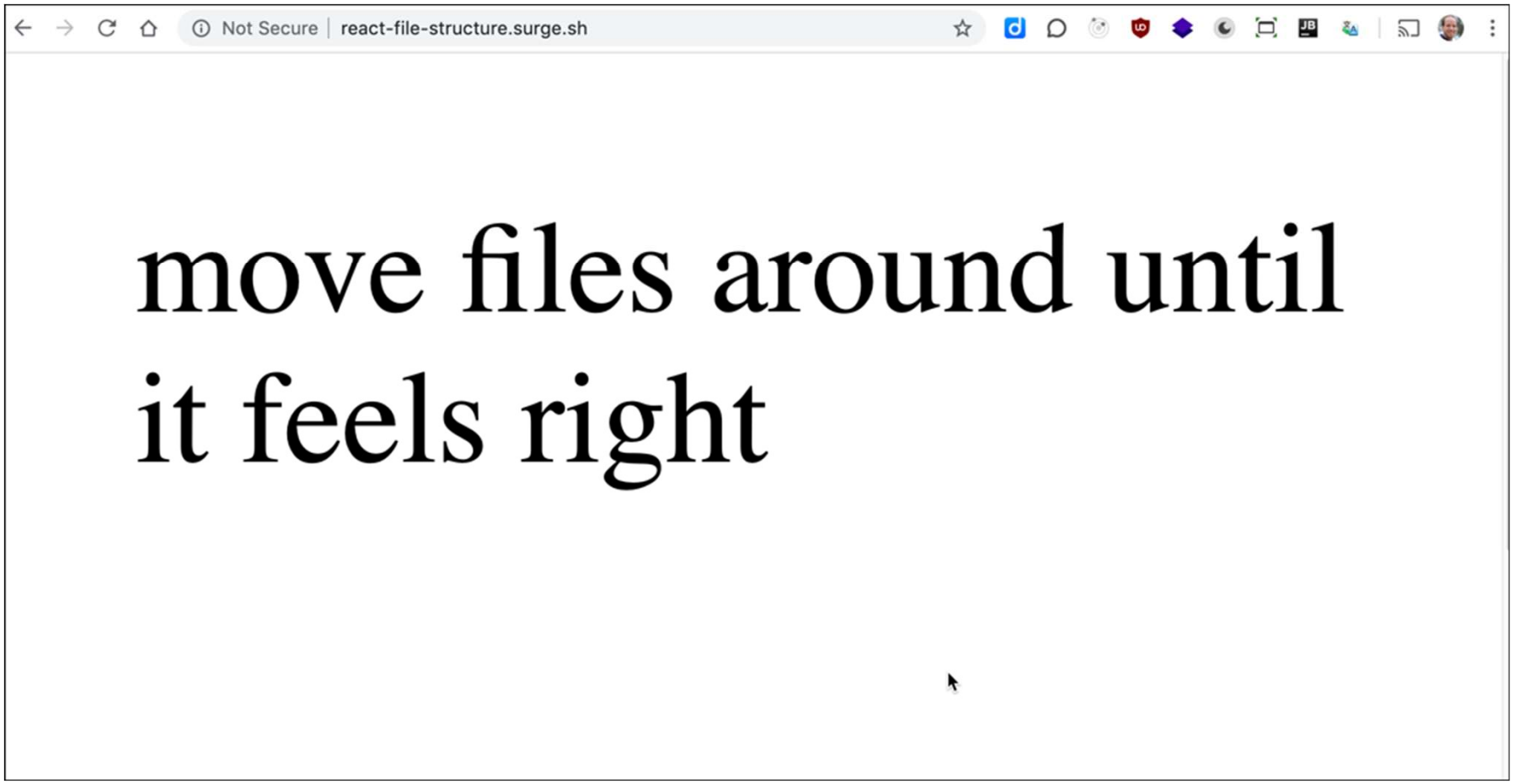
LEARN REACT > QUICK START >

Thinking in React

React can change how you think about the designs you look at and the apps you build. When you build a user interface with React, you will first break it apart into pieces called *components*. Then, you will describe the different visual states for each of your components. Finally, you will connect your components together so that the data flows through them. In this tutorial, we'll guide you through the thought process of building a searchable product data table with React.

https://react.dev/learn/thinking-in-react

How to structure your React project?



Dan Abramov: http://react-file-structure.surge.sh/

Some other opinions

React Docs Tutorial Blog Community

File Structure

Is there a recommended way to structure React projects?

React doesn't have opinions on how you put files into folders. That said there are a few common approaches popular in the ecosystem you may want to consider.

Grouping by features or routes

One common way to structure projects is locate CSS, JS, and tests together inside folders grouped by feature or route.

reactjs.org/docs/faq-structure.html

How I structure a React project

Maciek Chmura May 28 *Updated on Jun 05, 2019* · 2 min read

#javascript #react

There are many guides on how to structure web apps and React in particular.

- Move files into folders based on Smart/Dumb components.
- Organise them by the Header/Main/Footer
- Or throw everything to Components and combine them in Pages???

https://dev.to/maciekchmura/how-i-structure-a-react-project-3c2i

Advanced tips on structuring larger applications

sitepoint Premium Forum Developer Jobs Pricing Blog Search Login Start Free Trial

JavaScript / React / How to Organize a Large React Application and Make It Scale

Jack Franklin

November 2, 2020

How to Organize a Large React Application and Make It Scale

JavaScript React

Share this article

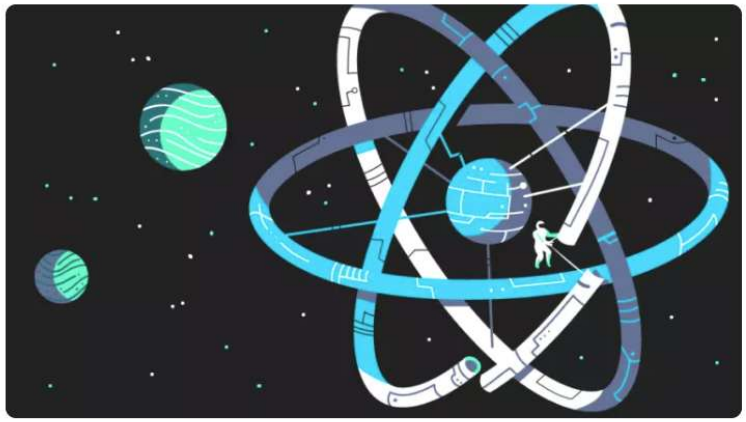


Table of Contents

Build Tools and Linting

Folder Structure

React Components

Prefer the jsx Extension over Capital Letters

One React Component per File

In this article, I'll discuss the approach I take when building and structuring large React applications. One of the best features of React is how it gets out of your way and is anything but descriptive when it comes to file structure. Therefore, you'll find a lot of questions on Stack Overflow and similar sites asking how to structure applications. This is a very opinionated topic, and there's no one right way. In this article, I'll talk you through the decisions I make when building React applications: picking tools, structuring files, and breaking components up into smaller pieces.

SitePoint Premium

Stay Relevant and Grow Your Career in Tech

- Hundreds of Courses and Books
- Curated Learning Paths
- Unlimited Access

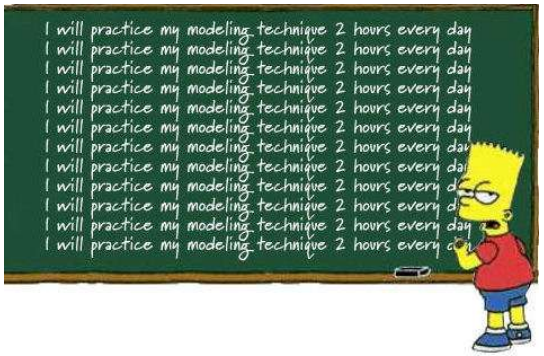
Privacy

www.sitepoint.com/organize-large-react-application

Workshop

- Create a new, blank React project (or go ahead with your previous project)
- Identify and study the files discussed in this section
  - Run the project: `npm start`
- Replace some content in `App` component with your own code.
- Add some images.
- Create your own (new) component.
  - Load it *instead* of `<App />`. OR:
  - Load it *inside* `<App />`
- Move component to a new `/components` folder.

  Make sure project still runs.
- Code example: ../100-helloworld



Checkpoint

- Introduction – overview of the front-end landscape
- React tooling – installation
- Hello World –the structure and architecture of React apps.
- Next up: Zooming in -
  - Using Hooks to handle component state